Rafael Angelo Christianto

HACK THE BOX SHERLOCK - DFIR

OPERATION BLACKOUT 2025: PHANTOM CHECK

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ ls
Microsoft-Windows-Powershell.evtx  PhantomCheck.zip  Windows-Powershell-Operational.evtx
```

After downloading and extracting out the files from PhantomCheck.zip we get 2 Windows Event Log files, both are Powershell logs, but one has the keyword Operational, which is more detailed and for more modern versions of Powershell.

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ chainsaw dump Windows-Powershell-Operational.evtx --json > powershell_operational.json

   By WithSecure Countercept (@FranticTyping, @AlexKornitzer)

[+] Dumping the contents of forensic artefacts from: Windows-Powershell-Operational.evtx (extensions: *)
[+] Loaded 1 forensic artefacts (15.1 MiB)
[+] Done
```

Then I first Chainsaw dumped the Windows-Powershell-Operational.evtx to powershell_operational.json.

1. Which WMI class did the attacker use to retrieve model and manufacturer information for virtualization detection?

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ strings powershell_operational.json | grep -i "WMI"
      "ScriptBlockText": "$Manufacturer = Get-WmiObject -Class Win32_ComputerSystem | select-object -expandproperty \"Manufacturer\"",
      "ContextInfo": "        Severity = Informational\r\n        Host Name = ConsoleHost\r\n        Host Version = 5.1.26100.2161\r\n        Hos
t ID = 0fad0cf8-6cb6-4657-86f7-655ec22eed9f\r\n        Host Application = C:\\Windows\\SysWOW64\\WindowsPowerShell\\v1.0\\powershell.exe\r\n
   Engine Version = 5.1.26100.2161\r\n        Runspace ID = 2aeeba59-d0f6-4ce7-b41c-e07625b3beec\r\n        Pipeline ID = 5\r\n        Command Na
me = Get-WmiObject\r\n        Command Type = Cmdlet\r\n        Script Name = \r\n        Command Path = \r\n        Sequence Number = 20\r\n
   User = DESKTOP-M3AKJSD\\User\r\n        Connected User = \r\n        Shell ID = Microsoft.PowerShell\r\n",
      "Payload": "CommandInvocation(Get-WmiObject): \"Get-WmiObject\"\r\nParameterBinding(Get-WmiObject): name=\"Class\"; value=\"Win32_ComputerS
ystem\"\r\nCommandInvocation(Select-Object): \"Select-Object\"\r\nParameterBinding(Select-Object): name=\"ExpandProperty\"; value=\"Manufacturer\
"\r\nParameterBinding(Select-Object): name=\"InputObject\"; value=\"\\\\DESKTOP-M3AKJSD\\root\\cimv2:Win32_ComputerSystem.Name=\"DESKTOP-M3AKJSD\
"\"\r\n"
      "ScriptBlockText": "$Model = Get-WmiObject -Class Win32_ComputerSystem | select-object -expandproperty \"Model\"",
      "ContextInfo": "        Severity = Informational\r\n        Host Name = ConsoleHost\r\n        Host Version = 5.1.26100.2161\r\n        Hos
t ID = 0fad0cf8-6cb6-4657-86f7-655ec22eed9f\r\n        Host Application = C:\\Windows\\SysWOW64\\WindowsPowerShell\\v1.0\\powershell.exe\r\n
   Engine Version = 5.1.26100.2161\r\n        Runspace ID = 2aeeba59-d0f6-4ce7-b41c-e07625b3beec\r\n        Pipeline ID = 10\r\n        Command N
ame = Get-WmiObject\r\n        Command Type = Cmdlet\r\n        Script Name = \r\n        Command Path = \r\n        Sequence Number = 28\r\n
    User = DESKTOP-M3AKJSD\\User\r\n        Connected User = \r\n        Shell ID = Microsoft.PowerShell\r\n",
      "Payload": "CommandInvocation(Get-WmiObject): \"Get-WmiObject\"\r\nParameterBinding(Get-WmiObject): name=\"Class\"; value=\"Win32_ComputerS
```

I first searched for the keyword WMI, and then immediately I spotted the keyword for this question which is "Manufacturer" and I spotted **Win32_ComputerSystem** is the class to get more information regarding the computer system.

2. Which WMI query did the attacker execute to retrieve the current temperature value of the machine?

```
      "ScriptBlockText": "Get-WmiObject -Query \"SELECT * FROM MSAcpi_ThermalZoneTemperature\" -ErrorAction SilentlyContinue",
      "ContextInfo": "        Severity = Informational\r\n        Host Name = ConsoleHost\r\n        Host Version = 5.1.26100.2161\r\n        Hos
t ID = 0fad0cf8-6cb6-4657-86f7-655ec22eed9f\r\n        Host Application = C:\\Windows\\SysWOW64\\WindowsPowerShell\\v1.0\\powershell.exe\r\n
   Engine Version = 5.1.26100.2161\r\n        Runspace ID = 2aeeba59-d0f6-4ce7-b41c-e07625b3beec\r\n        Pipeline ID = 21\r\n        Command N
ame = Get-WmiObject\r\n        Command Type = Cmdlet\r\n        Script Name = \r\n        Command Path = \r\n        Sequence Number = 54\r\n
    User = DESKTOP-M3AKJSD\\User\r\n        Connected User = \r\n        Shell ID = Microsoft.PowerShell\r\n",
      "Payload": "CommandInvocation(Get-WmiObject): \"Get-WmiObject\"\r\nParameterBinding(Get-WmiObject): name=\"Query\"; value=\"SELECT * FROM M
SAcpi_ThermalZoneTemperature\"\r\nParameterBinding(Get-WmiObject): name=\"ErrorAction\"; value=\"SilentlyContinue\"\r\nNonTerminatingError(Get-Wm
iObject): \"Invalid class \"MSAcpi_ThermalZoneTemperature\"\"\r\n"
```

Still the same command as the previous question number 1, and scrolled down a bit. Then I spotted a keyword again for this question which is "Temperature", and since it was the same format as the previous question and the question mentioned "query", so I took the whole SQL query, therefore the answer is **SELECT * FROM MSAcpi_ThermalZoneTemperature**.

3. The attacker loaded a PowerShell script to detect virtualization. What is the function name of the script?

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ strings powershell_operational.json | grep -i "virtual"
      "ScriptBlockText": "\r\n#requires -version 3.0\r\n\r\ntry { Microsoft.PowerShell.Core\\Set-StrictMode -Off } catch { }\r\n\r\n$script:MyMod
ule = $MyInvocation.MyCommand.ScriptBlock.Module\r\n\r\n$script:ClassName = 'ROOT/Microsoft/Windows/Storage/MSFT_Disk'\r\n$script:ClassVersion =
''\r\n$script:ModuleVersion = '1.0.0.0'\r\n$script:ObjectModelWrapper = [Microsoft.PowerShell.Cmdletization.Cim.CimCmdletAdapter]\r\n\r\n$script:
PrivateData = [System.Collections.Generic.Dictionary[string,string]]::new()\r\n\r\nMicrosoft.PowerShell.Core\\Export-ModuleMember -Function @()\r
\n        \r\n\r\nfunction __cmdletization_BindCommonParameters\r\n{\r\n    param(\r\n        $__cmdletization_objectModelWrapper,\r\n        $my
PSBoundParameters\r\n    )        \r\n                \r\n\r\n        if ($myPSBoundParameters.ContainsKey('CimSession')) { \r\n            $__cmd
letization_objectModelWrapper.PSObject.Properties['CimSession'].Value = $myPSBoundParameters['CimSession'] \r\n        }\r\n                    \
r\n\r\n        if ($myPSBoundParameters.ContainsKey('ThrottleLimit')) { \r\n            $__cmdletization_objectModelWrapper.PSObject.Properties['
ThrottleLimit'].Value = $myPSBoundParameters['ThrottleLimit'] \r\n        }\r\n                    \r\n\r\n        if ($myPSBoundParameters.Conta
insKey('AsJob')) { \r\n            $__cmdletization_objectModelWrapper.PSObject.Properties['AsJob'].Value = $myPSBoundParameters['AsJob'] \r\n
    }\r\n                \r\n\r\n}\r\n                \r\n\r\nfunction Get-Disk\r\n{\r\n    [CmdletBinding(DefaultParameterSetName='ByNumber
', PositionalBinding=$false)]\r\n    \r\n    [OutputType([Microsoft.Management.Infrastructure.CimInstance])]\r\n[OutputType('Microsoft.Management
```

I searched for the keyword "virtual" but found no appropriate function name that would check for a virtual machine. So then I filtered "VM".

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ strings powershell_operational.json | grep -i "VM"
      "ScriptBlockText": "if(((($Manufacturer.ToLower() -eq \"microsoft corporation\") -and ($Model.ToLower().contains(\"virtual\"))) -or ($Manufa
cturer.ToLower().contains(\"vmware\")) -or ($Model.ToLower() -eq \"virtualbox\")) {write-host \"Virtualization environment detected!\"} else {wri
te-host \"No virtualization environment detected!\"}",
      "ScriptBlockText": "function Check-VM\n{\n\n<# \n.SYNOPSIS \nNishang script which detects whether it is in a known virtual machine.\n \n.DE
SCRIPTION \nThis script uses known parameters or 'fingerprints' of Hyper-V, VMWare, Virtual PC, Virtual Box,\nXen and QEMU for detecting the envi
ronment.\n\n.EXAMPLE \nPS > Check-VM\n \n.LINK \nhttp://www.labofapenetrationtester.com/2013/01/quick-post-check-if-your-payload-is.html\nhttps:/
```

And as highlighted in the screenshot above, the name explains itself that it checks for a VM, and it is a function, so the answer is **Check-VM**.

4. Which registry key did the above script query to retrieve service details for virtualization detection?

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ strings powershell_operational.json | grep -i "Check-VM"
     "ScriptBlockText": "function Check-VM\n{\n\n<# \n.SYNOPSIS \nNishang script which detects whether it is in a known virtual machine.\n \n.DE
SCRIPTION \nThis script uses known parameters or 'fingerprints' of Hyper-V, VMWare, Virtual PC, Virtual Box,\nXen and QEMU for detecting the envi
ronment.\n\n.EXAMPLE \nPS > Check-VM\n \n.LINK \nhttp://www.labofapenetrationtester.com/2013/01/quick-post-check-if-your-payload-is.html\nhttps:/
/github.com/samratashok/nishang\n\n.NOTES \nThe script draws heavily from checkvm.rb post module from msf.\nhttps://github.com/rapid7/metasploit-
framework/blob/master/modules/post/windows/gather/checkvm.rb\n#> \n    [CmdletBinding()] Param()\n    $ErrorActionPreference = \"SilentlyContinue
\"\n    #Hyper-V\n    $hyperv = Get-ChildItem HKLM:\\SOFTWARE\\Microsoft\n    if (($hyperv -match \"Hyper-V\") -or ($hyperv -match \"VirtualMachi
ne\"))\n        {\n            $hypervm = $true\n        }\n\n    if (!$hypervm)\n        {\n            $hyperv = Get-ItemProperty hklm:\\HARDWA
RE\\DESCRIPTION\\System -Name SystemBiosVersion\n            if ($hyperv -match \"vrtual\")\n                {\n                $hypervm = $t
rue\n                }\n        }\n    \n    if (!$hypervm)\n        {\n            $hyperv = Get-ChildItem HKLM:\\HARDWARE\\ACPI\\FADT\n
    if ($hyperv -match \"vrtual\")\n                {\n                $hypervm = $true\n                }\n        }\n            \n    if (
!$hypervm)\n        {\n            $hyperv = Get-ChildItem HKLM:\\HARDWARE\\ACPI\\RSDT\n            if ($hyperv -match \"vrtual\")\n
    {\n                $hypervm = $true\n                }\n        }\n\n    if (!$hypervm)\n        {\n            $hyperv = Get-ChildItem HK
LM:\\SYSTEM\\ControlSet001\\Services\n            if (($hyperv -match \"vmicheartbeat\") -or ($hyperv -match \"vmicvss\") -or ($hyperv -match \"v
```

I have tried to filter out for registry keys by using other keywords, but even in the screenshot above shows there are several HKLM registry keys but according to this question, it asked for which script query for getting the service details, and it is **HKLM:\SYSTEM\ControlSet001\Services.**

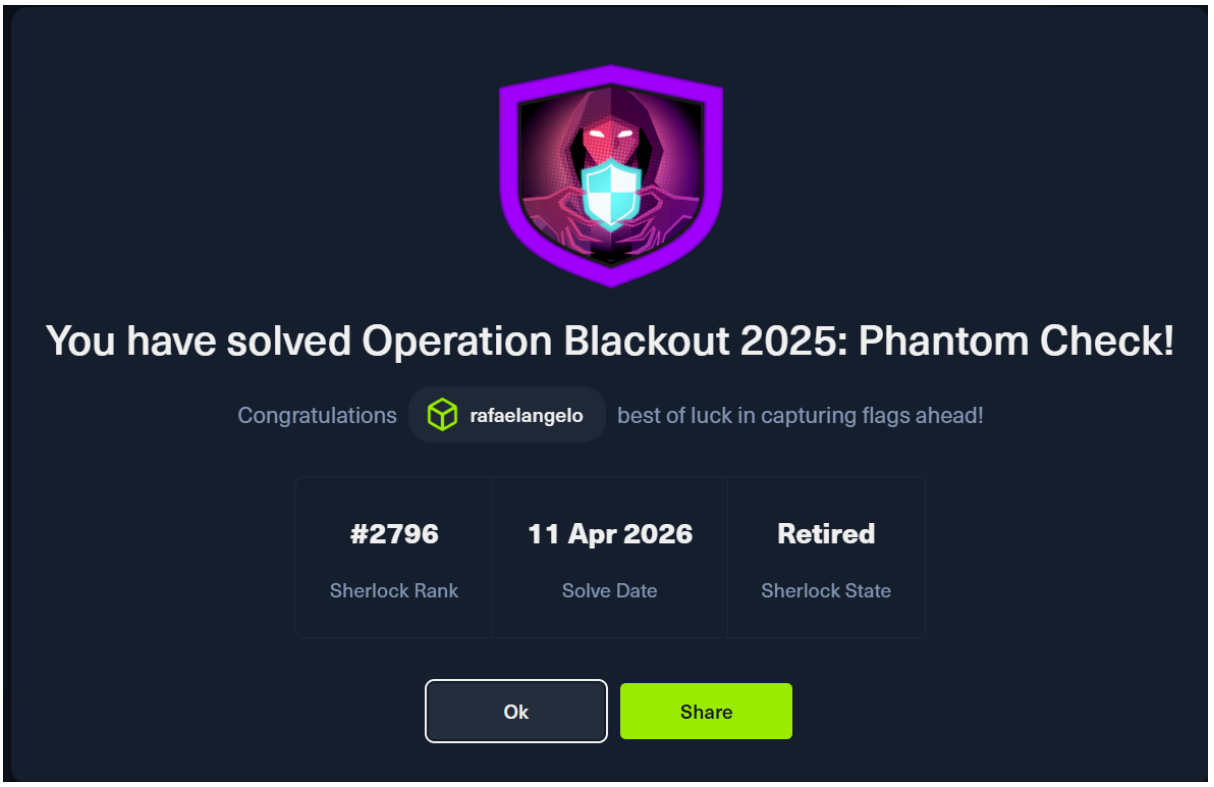
5. The VM detection script can also identify VirtualBox. Which processes is it comparing to determine if the system is running VirtualBox?

```
TEM\\ControlSet001\\Services\n            if (($vpc -match \"vpc-s3\") -or ($vpc -match \"vpcuhub\") -or ($vpc -match \"msvmmouf\"))\n
    {\n                $vpcvm = $true\n            }\n        }\n\n    if ($vpcvm)\n        {\n    \n        \"This is a Virtual PC.\"
n    \n        }\n\n\n    #Virtual Box\n\n    $vb = Get-Process\n    if (($vb -eq \"vboxservice.exe\") -or ($vb -match \"vboxtray.exe\"))\n
  {\n    \n        $vbvm = $true\n    \n        }\n    if (!$vbvm)\n        {\n            $vb = Get-ChildItem HKLM:\\HARDWARE\\ACPI\\FADT\n
      if ($vb -match \"vbox_\")\n            {\n                $vbvm = $true\n            }\n        }\n\n    if (!$vbvm)\n
{\n            $vb = Get-ChildItem HKLM:\\HARDWARE\\ACPI\\RSDT\n            if ($vb -match \"vbox_\")\n            {\n            $v
vm = $true\n            }\n        }\n\n    \n    if (!$vbvm)\n        {\n            $vb = Get-Childitem hklm:\\hardware\\devicemap\\scsi -
ecurse | gp -Name identifier\n            if ($vb -match \"vbox\")\n            {\n                $vbvm = $true\n            }\n
    }\n\n\n\n    if (!$vbvm)\n        {\n            $vb = Get-ItemProperty hklm:\\HARDWARE\\DESCRIPTION\\System -Name SystemBiosVersion\n
    if ($vb -match \"vbox\")\n            {\n                $vbvm = $true\n            }\n        }\n  \n\n    if (!$vbvm)\n
  {\n            $vb = Get-ChildItem HKLM:\\SYSTEM\\ControlSet001\\Services\n            if (($vb -match \"VBoxMouse\") -or ($vb -match \"VBoxGu
st\") -or ($vb -match \"VBoxService\") -or ($vb -match \"VBoxSF\"))\n            {\n                $vbvm = $true\n            }\n
```

Still under the same command as before, and scrolling down a little, since it is still related to Check-VM and I saw 2 virtual box related executables, and it fits to this question to check whether it is virtual box, so there are 2 answers for this question and they are **vboxservice.exe, vboxtray.exe**.

6. The VM detection script prints any detection with the prefix 'This is a'. Which two virtualization platforms did the script detect?

```
┌──(rafael㉿raf)-[~/HTB_SHERLOCKS/PhantomCheck]
└─$ strings powershell_operational.json | grep -i "this is a"
      "ScriptBlockText": "function Check-VM\n{\n\n<# \n.SYNOPSIS \nNishang script which detects whether it is in a known virtual machine.\n \n.DE
SCRIPTION \nThis script uses known parameters or 'fingerprints' of Hyper-V, VMWare, Virtual PC, Virtual Box,\nXen and QEMU for detecting the envi
ronment.\n\n.EXAMPLE \nPS > Check-VM\n \n.LINK \nhttp://www.labofapenetrationtester.com/2013/01/quick-post-check-if-your-payload-is.html\nhttps:/
/github.com/samratashok/nishang\n\n.NOTES \nThe script draws heavily from checkvm.rb post module from msf.\nhttps://github.com/rapid7/metasploit-
framework/blob/master/modules/post/windows/gather/checkvm.rb\n#> \n    [CmdletBinding()] Param()\n    $ErrorActionPreference = \"SilentlyContinue
\"\n    #Hyper-V\n    $hyperv = Get-ChildItem HKLM:\\SOFTWARE\\Microsoft\n    if (($hyperv -match \"Hyper-V\") -or ($hyperv -match \"VirtualMachi
ne\"))\n        {\n            $hypervm = $true\n        }\n\n    if (!$hypervm)\n        {\n            $hyperv = Get-ItemProperty hklm:\\HARDWA
RE\\DESCRIPTION\\System -Name SystemBiosVersion\n            if ($hyperv -match \"vrtual\")\n                {\n                    $hypervm = $t
rue\n                }\n        }\n    \n    if (!$hypervm)\n        {\n            $hyperv = Get-ChildItem HKLM:\\HARDWARE\\ACPI\\FADT\n
    if ($hyperv -match \"vrtual\")\n                {\n                    $hypervm = $true\n                }\n        }\n            \n    if (
!$hypervm)\n        {\n            $hyperv = Get-ChildItem HKLM:\\HARDWARE\\ACPI\\RSDT\n            if ($hyperv -match \"vrtual\")\n
    {\n                    $hypervm = $true\n                }\n        }\n\n    if (!$hypervm)\n        {\n            $hyperv = Get-ChildItem HK
LM:\\SYSTEM\\ControlSet001\\Services\n            if (($hyperv -match \"vmicheartbeat\") -or ($hyperv -match \"vmicvss\") -or ($hyperv -match \"v
micshutdown\") -or ($hyperv -match \"vmiexchange\"))\n                {\n                    $hypervm = $true\n                }\n        }\n    \
n    if ($hypervm)\n        {\n    \n            \"This is a Hyper-V machine.\"\n    \n        }\n\n    #VMWARE\n\n    $vmware = Get-ChildItem H
KLM:\\SYSTEM\\ControlSet001\\Services\n    if (($vmware -match \"vmdebug\") -or ($vmware -match \"vmmouse\") -or ($vmware -match \"VMTools\") -or
 ($vmware -match \"VMMEMCTL\"))\n        {\n            $vmwarevm = $true\n        }\n\n    if (!$vmwarevm)\n        {\n            $vmware = Get
-ItemProperty hklm:\\HARDWARE\\DESCRIPTION\\System\\BIOS -Name SystemManufacturer\n            if ($vmware -match \"vmware\")\n                {\
n                    $vmwarevm = $true\n                }\n        }\n    \n    if (!$vmwarevm)\n        {\n            $vmware = Get-Childitem h
klm:\\hardware\\devicemap\\scsi -recurse | gp -Name identifier\n            if ($vmware -match \"vmware\")\n                {\n
  $vmwarevm = $true\n                }\n        }\n\n    if (!$vmwarevm)\n        {\n            $vmware = Get-Process\n            if (($vmware
-eq \"vmwareuser.exe\") -or ($vmware -match \"vmwaretray.exe\"))\n                {\n                    $vmwarevm = $true\n                }\n
      }\n\n    if ($vmwarevm)\n        {\n    \n            \"This is a VMWare machine.\"\n    \n        }\n    \n    #Virtual PC\n\n    $vpc =
Get-Process\n    if (($vpc -eq \"vmusrvc.exe\") -or ($vpc -match \"vmsrvc.exe\"))\n        {\n        $vpcvm = $true\n        }\n\n    if (!$vpcv
m)\n        {\n            $vpc = Get-Process\n            if (($vpc -eq \"vmwareuser.exe\") -or ($vpc -match \"vmwaretray.exe\"))\n
    {\n                    $vpcvm = $true\n                }\n        }\n\n    if (!$vpcvm)\n        {\n            $vpc = Get-ChildItem HKLM:\\SY
STEM\\ControlSet001\\Services\n            if (($vpc -match \"vpc-s3\") -or ($vpc -match \"vpcuhub\") -or ($vpc -match \"msvmmouf\"))\n
```

```
      "Payload": "CommandInvocation(Out-Default): \"Out-Default\"\r\nParameterBinding(Out-Default): name=\"InputObject\"; value=\"This is a Hyper
-V machine.\"\r\nParameterBinding(Out-Default): name=\"InputObject\"; value=\"This is a VMWare machine.\"\r\n"
```

I filtered for “This is a” as a string for better visibility, and we can see there’s only 2 in the payload attribute which are **Hyper-V, Vmware**.